REST Endpoints, FastAPI, and LangServe — Complete Notes

1 What Is a REST Endpoint?

REST = Representational State Transfer

It is a way applications communicate over HTTP.

An **endpoint** is simply a URL that performs an action.

Example:

POST /chain/invoke

GET /chain/input_schema

Each endpoint:

- Receives a request
- Processes it
- Returns a response

Think of it as a function exposed over the internet.

2 What Is an API?

API = Application Programming Interface

It allows:

- One app to talk to another
- Frontend to talk to backend

- Postman to talk to your server
- Browser to talk to your AI app

Instead of calling a Python function directly, you send an HTTP request.

3 GET vs POST (Very Important)

◇ GET

Used to retrieve information.

- No request body
- Does not modify anything

Example: GET /chain/input_schema

Returns metadata about what inputs are required.

◇ POST

Used to send data to the server.

- Has a request body (JSON)
- Triggers processing

Example: POST /chain/invoke

Body: { "language": "French", "text": "Hello" }

Response: { "output": "Bonjour" }

4 What Is FastAPI?

FastAPI is a Python web framework for building APIs.

It:

- Creates routes (endpoints)
- Handles JSON automatically
- Generates documentation automatically
- Validates input/output
- Runs using Uvicorn (ASGI server)

5 What Is Uvicorn?

Uvicorn is a server that runs your FastAPI app.

When you do:

```
uvicorn.run(app, host="127.0.0.1", port=8000)
```

It:

- Starts a web server
- Listens for incoming requests
- Sends responses back

6 What Is LangServe?

LangServe converts LangChain chains into REST APIs automatically.

When you write:

```
add_routes(app, chain, path="/chain")
```

LangServe creates:

- POST /chain/invoke
- POST /chain/batch
- GET /chain/input_schema
- GET /chain/output_schema

You do not manually write route functions. It wraps your chain automatically.

7 How Is Documentation Generated Automatically?

FastAPI automatically generates Swagger UI documentation.

URL: http://127.0.0.1:8000/docs

It inspects:

- Your routes
- Input schema
- Output schema
- Pydantic models
- Chain structure

Then generates:

- Interactive documentation
- Test interface
- Input/output schemas
- Example JSON

This is automatic because FastAPI uses:

- Type hints
- Pydantic models
- Route metadata

LangServe provides schema info from your chain, so FastAPI can document it.

8 What Changes When Chain Changes?

If you change:

```
chain = prompt | model | StrOutputParser()
```

Output schema = string

If you change:

chain = prompt | model | PydanticOutputParser()

Output schema = structured JSON

FastAPI automatically updates /docs accordingly.

9️⃣ Full System Flow (Step-by-Step)

User / Postman / Frontend

↓

HTTP Request (POST /chain/invoke)

↓

FastAPI Route

↓

LangServe add_routes

↓

LangChain Chain

↓

Prompt → LLM → Parser

↓

Final Output

↓

JSON Response Returned

🔟 Libraries Used

Core Libraries

- fastapi → API framework
- uvicorn → server runner
- langchain-core → prompts + parsers
- langchain-groq → Groq LLM integration
- langserve → converts chains to REST APIs
- dotenv → load environment variables
- os → access API keys

1️⃣1️⃣ What Is Deployment?

Local run: http://127.0.0.1:8000

Only your computer can access it.

Deployment means: Hosting your FastAPI server on the internet.

Example platforms:

- AWS
- GCP
- Azure
- Render
- Railway
- Docker container

After deployment: https://yourapp.com

Now:

- Anyone can access it
- Frontend apps can use it

- It runs 24/7

1️⃣2️⃣ Big Concept Shift

Notebook = Experimentation

FastAPI + LangServe = Backend API

Deployment = Hosting backend online

You are moving from: "Running models locally"

To: "Building production-ready AI services"